

Perforce P4Prometheus - Overview

Perforce Professional Services

Version v2026.1, 2026-09-15

Table of Contents

Preface	2
1. Installation	3
2. Support Status	4
3. Overview	5
4. Grafana Dashboards	6
5. Metrics Available	9
5.1. p4prometheus Metrics	9
5.2. p4metrics Metrics	9
5.3. Lock Metrics	9
6. Related Tools	11

[Support]

Preface

P4Prometheus integrates Perforce Helix Core Server (p4d) with [Prometheus](#) and related monitoring tools. It collects real-time metrics from p4d logs, p4 monitor, and related sources, then exposes them for Prometheus collection and Grafana dashboards. The metrics can also drive system alerting.

The p4prometheus service continuously parses p4d logs and writes Prometheus textfile-collector metrics. Other project components interrogate p4d and associated logs for supplementary metrics.

Chapter 1. Installation

Use the automated scripts for new installations and upgrades. The canonical guide is [P4Prometheus Installation](#).

It covers monitoring-server, Helix Core server, and node-exporter-only installations; upgrades; dedicated data volumes; air-gapped installations; state files; and manual component installation where scripts cannot be used.

Chapter 2. Support Status

This is a Community Supported Perforce tool.

Chapter 3. Overview

P4Prometheus is part of a monitoring solution made of these components:

- [Prometheus](#): time-series metrics management.
- [VictoriaMetrics](#): optional, Prometheus-compatible long-term metrics storage.
- [Grafana](#): metrics dashboards and visualization.
- [node_exporter](#): Linux host metrics and textfile collection.
- [windows_exporter](#): Windows host metrics.
- [Alertmanager](#): alert routing, grouping, and de-duplication.

Project components:

- [p4prometheus](#): a released binary that parses Helix Core logs.
- [p4metrics](#): supplementary Helix Core and SDP-compatible metrics.
- [monitor_metrics.py](#) and [monitor_wrapper.sh](#): Linux lock monitoring using `lslocks` and `p4 monitor show`.
- [p4logtail](#) and [p4plogtail](#): completed-command log tails with JSON output.

P4Prometheus uses [go-libp4dlog](#) for log parsing.

The installation and upgrade scripts support standard installation, dedicated data volumes (`-d /data`), offline assets (`--local-tarballs-dir`), SDP-safe configuration in `/p4/common/site/config/`, hostname-specific HMS fleet configuration, and stateful non-destructive upgrades. See [the installation guide](#) for details.

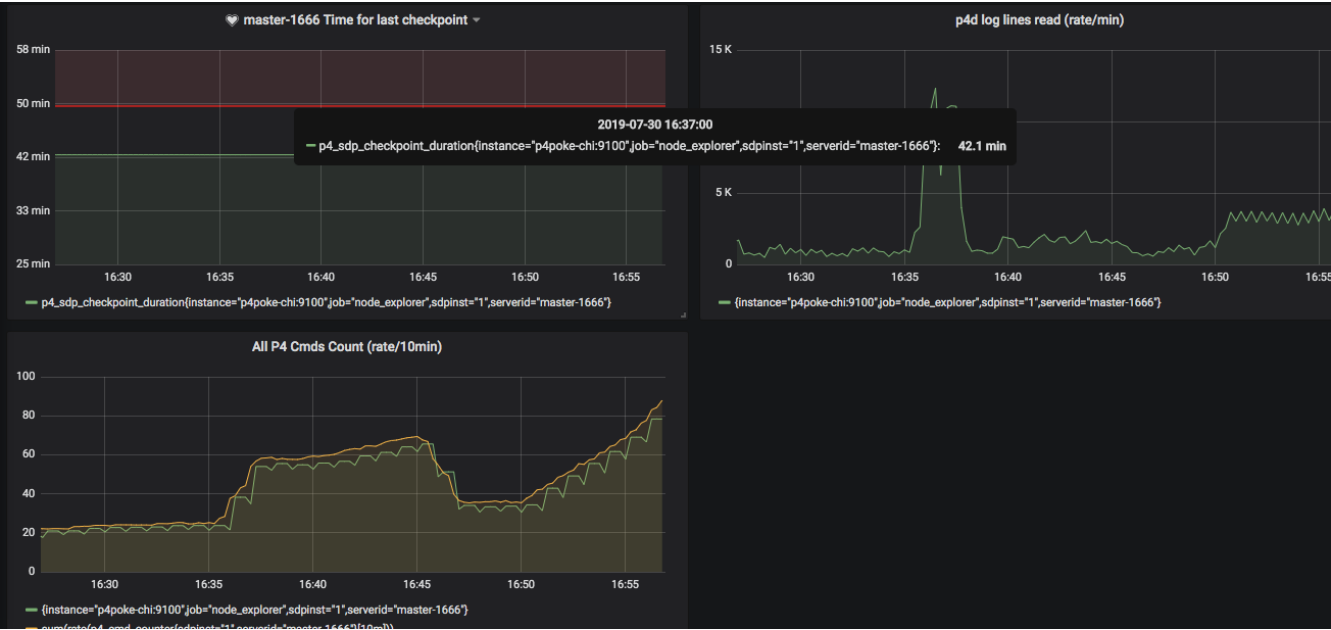
[Prometheus overview](#)

[Prometheus architecture]

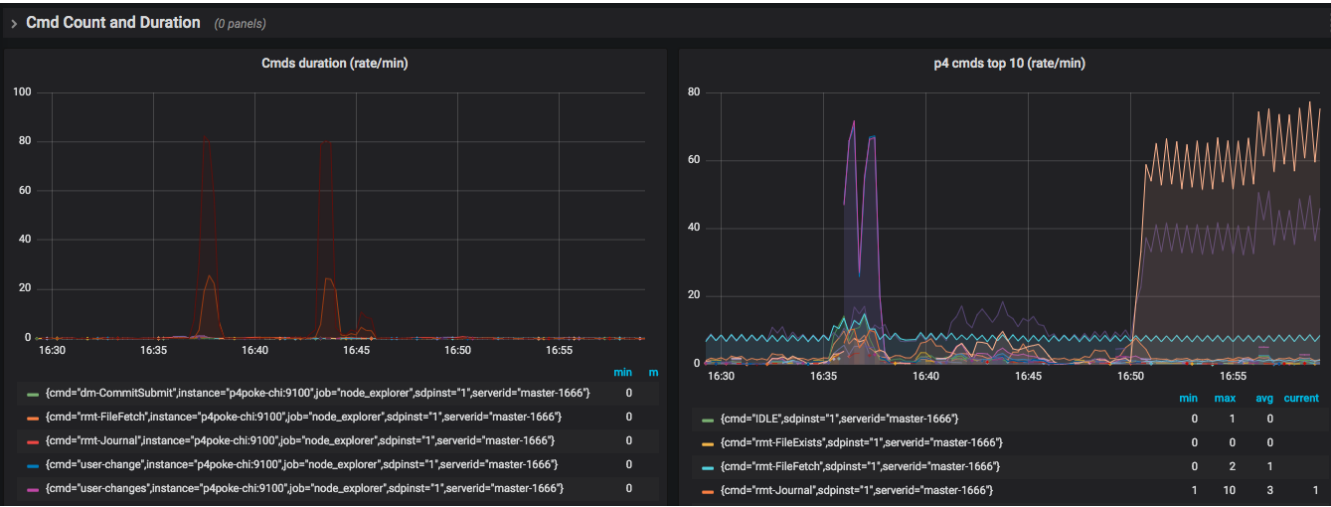
Chapter 4. Grafana Dashboards

When configured, Grafana dashboards provide command, replication, lock, and host-level views.

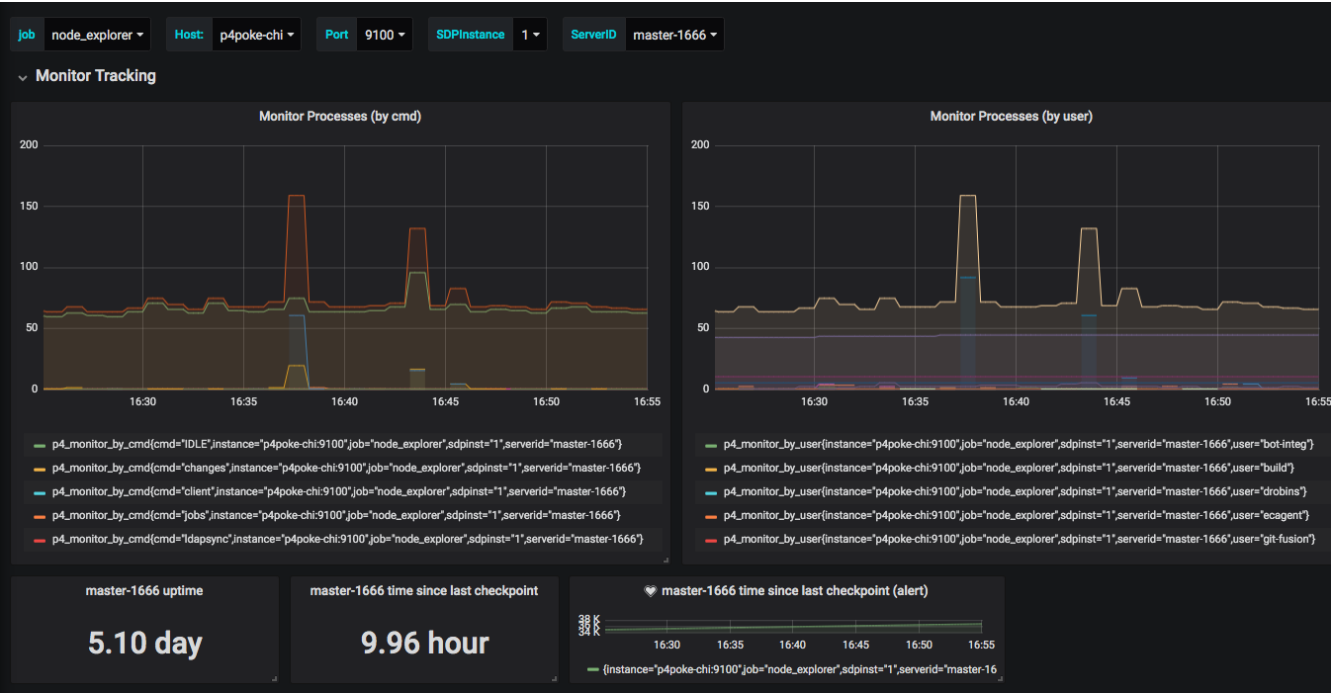
Commands Summary



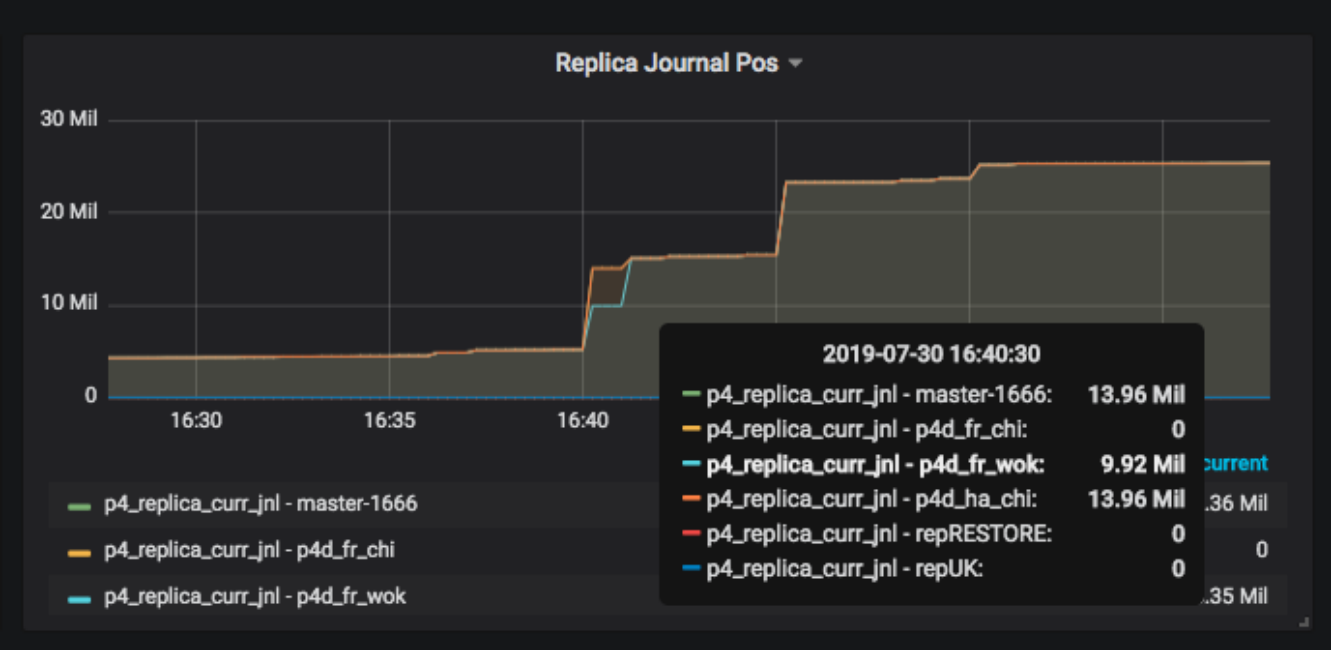
Command Rates and Durations



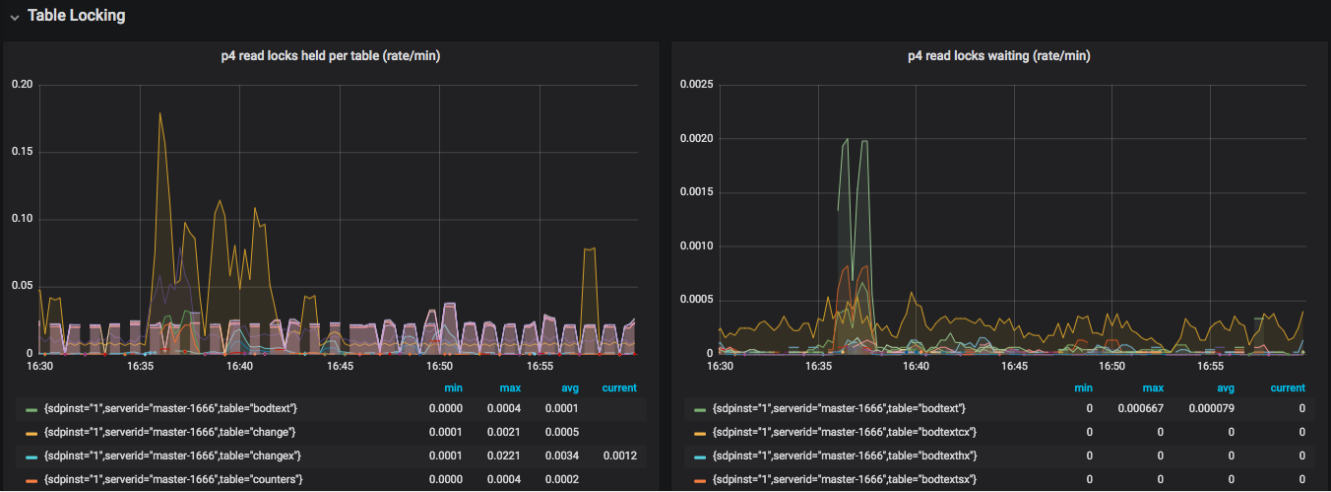
Active Commands



Replication Status



Read and Write Locks



Grafana dashboards can define alerts; Alertmanager processes their notifications. The installation guide includes setup and validation guidance, and [Runbook Alert Handling](#) provides a customizable operational runbook.

Chapter 5. Metrics Available

All principal metrics carry `serverid` and, for SDP installations, `sdpinst` labels. The README remains the exhaustive metric-name reference: [metrics available](#).

5.1. p4prometheus Metrics

The primary log parser reports command throughput, active commands, resource use, synchronization activity, command error counts, command durations, and lock wait/held time.

Metric family	Examples
Parser health and throughput	<code>p4_prom_log_lines_read</code> , <code>p4_prom_cmds_processed</code> , <code>p4_prom_cmds_pending</code>
Command activity	<code>p4_cmd_running</code> , <code>p4_cmd_counter</code> , <code>p4_cmd_cumulative_seconds</code>
Resource use	<code>p4_prom_cpu_user</code> , <code>p4_prom_cpu_system</code> , <code>p4_prom_memory</code>
Command dimensions	<code>p4_cmd_user_counter</code> , <code>p4_cmd_ip_counter</code> , <code>p4_cmd_replica_counter</code> , <code>p4_cmd_program_counter</code>
Lock timings	<code>p4_total_read_wait_seconds</code> , <code>p4_total_read_held_seconds</code> , <code>p4_total_write_wait_seconds</code> , <code>p4_total_write_held_seconds</code>

5.2. p4metrics Metrics

`p4metrics` replaces the legacy `monitor_metrics.sh` metrics while retaining compatible names and labels. It reports active processes, memory, server identity and uptime, replication, journal/log rotation, license state, SDP job results, Swarm status, and optional memory-limit enforcement.

Metric family	Examples
Process monitoring	<code>p4_monitor_by_cmd</code> , <code>p4_monitor_by_state</code> , <code>p4_monitor_by_user</code> , <code>p4_monitor_max_cmd_time</code>
Memory and limits	<code>p4_active_memory_by_cmd</code> , <code>p4_active_memory_by_user</code> , <code>p4_memlimit_kill_candidates</code> , <code>p4_memlimit_kills_total</code>
Replication	<code>p4_pull_queue_total</code> , <code>p4_pull_replica_lag</code> , <code>p4_pull_replication_error</code>
Server and license	<code>p4_server_uptime</code> , <code>p4_p4d_build_info</code> , <code>p4_license_expires</code> , <code>p4_licensed_user_count</code>
SDP jobs and logs	<code>p4_sdp_checkpoint_error</code> , <code>p4_sdp_verify_errors</code> , <code>p4_journal_size</code> , <code>p4_log_size</code>
Swarm	<code>p4_swarm_error</code> , <code>p4_swarm_tasks</code> , <code>p4_swarm_workers</code> , <code>p4_swarm_version</code>

5.3. Lock Metrics

Linux lock monitoring requires `lslocks`. `monitor_wrapper.sh` invokes `monitor_metrics.py`, which

relates operating-system locks to `p4_monitor_show` results. It reports:

- `p4_locks_db_read` and `p4_locks_db_write`
- `p4_locks_cliententity_read` and `p4_locks_cliententity_write`
- `p4_locks_meta_read` and `p4_locks_meta_write`
- `p4_locks_cmds_blocked`

The detailed lock-monitoring installation and troubleshooting information is in [P4Prometheus Installation](#).

Chapter 6. Related Tools

- [p4logtail](#)
- [p4plogtail](#)
- [p4erroranalyzer](#)
- [logresourcepressure](#)